Polyglot Atlas: Claude Code brief

Nov 11, 2026 · @Esa

Build an offline iPhone app holding a full, Tae Kim-style grammar guide for each of 100 languages, written from the ground up and verified against real sources. This brief is the complete instruction set: read it all before writing anything.

1. Mission and non-negotiables

The product is a personal iPhone app called Polyglot Atlas. The owner opens it when bored, scrolls or shuffles through 100 languages, picks one, and reads a complete grammar guide that teaches the language's logic from zero. He is the only user. There are no accounts, no backend, no ads and no analytics.

The owner is an experienced language learner (Spanish, Korean, Japanese, French, some Arabic script and Moroccan Darija). He learns by understanding why a rule exists, not by memorising tables or drilling. Write for an intelligent adult who has never seen the language but will not tolerate hand-waving.

Rules that never bend:

1. **Content before app.** The guides are the product. The app is a reader. Never spend a session polishing UI while guides are unwritten.
2. **Never invent language data.** Every form, paradigm, example sentence and gloss must be traceable to a named source or be marked with a confidence label (section 5). An honest gap beats a confident fabrication.
3. **Full size means full size.** A depth-A guide is 40,000 to 70,000 words across 60 to 100 lessons. Do not summarise, compress or stop at an overview. If a session runs short, finish fewer lessons properly.
4. **Explain before use.** No example may contain a grammatical feature that has not been taught, unless it is glossed and flagged as a preview.
5. **Every example is glossed.** Original script, transliteration, morpheme-by-morpheme gloss, natural translation, and a literal translation where it differs.
6. **Resumable at all times.** All state lives in files in the repo (section 6). Any session can die at any moment and the next one continues without loss.
7. **Fully offline.** All guide content is stored on the phone and works with no connection.
8. **No copying.** Do not reproduce text from Tae Kim's guide or any textbook or grammar. Take the method, write original prose and original examples, and cite attested examples to their source.

9. **Ask nothing, log everything.** The owner is asleep. Take the most reasonable reading, record the decision in `DECISIONS.md`, and continue. Stop only for things that cost money or need his Apple ID.

2. The 100 languages

The list is fixed at 100: 71 living and 29 dead or classical. Spanish, French and Portuguese are deliberately excluded. Do not add, drop or merge languages without logging it in `DECISIONS.md`.

Depth classes set the target size. **A** = 40,000 to 70,000 words, 60 to 100 lessons. **B** = 25,000 to 40,000 words, because published documentation is thinner. **C** = as much as the evidence supports, with open problems presented as open.

#	Language	Group	Script taught	Depth	Notes
1	Italian	Major living	Latin	A	
2	German	Major living	Latin	A	
3	Dutch	Major living	Latin	A	
4	Russian	Major living	Cyrillic	A	
5	Polish	Major living	Latin	A	
6	Modern Greek	Major living	Greek	A	
7	Mandarin	Major living	Hanzi + pinyin	A	Simplified main, traditional noted
8	Cantonese	Major living	Hanzi + Jyutping	A	Spoken Cantonese, not written Mandarin
9	Japanese	Major living	Kana + kanji	A	
10	Korean	Major living	Hangul	A	Appendix: the register of muga (shaman songs)
11	Arabic (Modern Standard)	Major living	Arabic	A	
12	Moroccan Darija	Major living	Arabizi + Arabic	A	Arabizi first (3, 7, 9); flag MSA forms and regional variation
13	Modern Hebrew	Major living	Hebrew	A	
14	Persian	Major living	Perso-Arabic	A	
15	Turkish	Major living	Latin	A	
16	Hindi-Urdu	Major living	Devanagari + Nastaliq	A	One grammar, two scripts
17	Bengali	Major living	Bengali	A	

#	Language	Group	Script taught	Depth	Notes
18	Tamil	Major living	Tamil	A	Note spoken vs literary split
19	Indonesian	Major living	Latin	A	
20	Vietnamese	Major living	Latin (Quoc ngu)	A	
21	Thai	Major living	Thai	A	
22	Swahili	Major living	Latin	A	
23	Basque	Europe, less common	Latin	A	
24	Welsh	Europe, less common	Latin	A	
25	Irish	Europe, less common	Latin	A	
26	Icelandic	Europe, less common	Latin	A	
27	Finnish	Europe, less common	Latin	A	
28	Hungarian	Europe, less common	Latin	A	
29	Lithuanian	Europe, less common	Latin	A	
30	Albanian	Europe, less common	Latin	A	
31	Romanian	Europe, less common	Latin	A	
32	Serbo-Croatian	Europe, less common	Latin + Cyrillic	A	
33	Maltese	Europe, less common	Latin	A	
34	Georgian	Caucasus and Central Asia	Mkhedruli	A	
35	Armenian (Eastern)	Caucasus and Central Asia	Armenian	A	
36	Chechen	Caucasus and Central Asia	Cyrillic	B	
37	Kazakh	Caucasus and Central Asia	Cyrillic + Latin	A	
38	Mongolian (Khalkha)	Caucasus and Central Asia	Cyrillic	A	
39	Pashto	Caucasus and Central Asia	Perso-Arabic	A	

#	Language	Group	Script taught	Depth	Notes
40	Kurdish (Kurmanji)	Caucasus and Central Asia	Latin (Hawar)	A	
41	Telugu	Asia and Pacific	Telugu	A	
42	Burmese	Asia and Pacific	Burmese	A	
43	Khmer	Asia and Pacific	Khmer	A	
44	Tagalog	Asia and Pacific	Latin	A	
45	Balinese	Asia and Pacific	Balinese + Latin	B	Speech levels; appendix on reading lontar and Kawi vocabulary
46	Malagasy	Asia and Pacific	Latin	A	
47	Hawaiian	Asia and Pacific	Latin	A	
48	Maori	Asia and Pacific	Latin	A	
49	Tok Pisin	Asia and Pacific	Latin	A	
50	Yoruba	Africa	Latin	A	
51	Igbo	Africa	Latin	A	
52	Hausa	Africa	Latin (Boko)	A	
53	Zulu	Africa	Latin	A	
54	Amharic	Africa	Ethiopic	A	
55	Somali	Africa	Latin	A	
56	Wolof	Africa	Latin	A	
57	Tamazight	Africa	Tifinagh + Latin	B	Name the variety chosen in the overview
58	Taa (!Xoo)	Africa	Latin + click symbols	B	
59	Quechua (Southern)	Americas	Latin	A	Part on colonial Quechua spelling and texts
60	Guarani	Americas	Latin	A	
61	Yucatec Maya	Americas	Latin	A	Part on colonial texts and orthography
62	K'iche'	Americas	Latin	A	Part on colonial K'iche' (Popol Vuh)
63	Lakota	Americas	Latin	A	
64	Navajo	Americas	Latin	A	
65	Cherokee	Americas	Syllabary + Latin	A	Published sources only; respect community guidance on restricted material

#	Language	Group	Script taught	Depth	Notes
66	Greenlandic (Kalaallisut)	Americas	Latin	A	
67	Mapudungun	Americas	Latin	B	
68	Ainu	Isolates and unusual	Latin + katakana	B	
69	Burushaski	Isolates and unusual	Latin	B	
70	Dyirbal	Isolates and unusual	Latin	B	
71	Warlpiri	Isolates and unusual	Latin	B	
72	Latin	Dead and classical	Latin	A	
73	Ancient Greek (Attic)	Dead and classical	Greek	A	
74	Sanskrit	Dead and classical	Devanagari + IAST	A	
75	Pali	Dead and classical	Latin (IAST)	A	
76	Classical Chinese	Dead and classical	Hanzi	A	
77	Old English	Dead and classical	Latin	A	
78	Old Norse	Dead and classical	Latin, runes appendix	A	
79	Gothic	Dead and classical	Gothic + Latin	A	
80	Old Irish	Dead and classical	Latin	A	
81	Old Church Slavonic	Dead and classical	Cyrillic, Glagolitic appendix	A	
82	Biblical Hebrew	Dead and classical	Hebrew	A	
83	Classical Syriac	Dead and classical	Syriac	A	
84	Jewish Babylonian Aramaic	Dead and classical	Hebrew square script	B	Built toward reading incantation bowls
85	Mandaic	Dead and classical	Mandaic	B	

#	Language	Group	Script taught	Depth	Notes
86	Ge'ez	Dead and classical	Ethiopic	A	Built toward reading magic scrolls
87	Akkadian	Dead and classical	Transliteration, cuneiform intro	A	Old Babylonian base
88	Sumerian	Dead and classical	Transliteration, cuneiform intro	B	Mark contested analyses
89	Middle Egyptian	Dead and classical	Hieroglyphs + transliteration	A	
90	Coptic (Sahidic)	Dead and classical	Coptic	A	Appendix on magical texts
91	Old Nubian	Dead and classical	Old Nubian (Coptic-based)	B	
92	Hittite	Dead and classical	Transliteration	B	
93	Classical Maya	Dead and classical	Glyphs + transliteration	B	Teach how the script works, not only the language
94	Classical Nahuatl	Dead and classical	Latin	A	
95	Tocharian B	Dead and classical	Transliteration	B	
96	Etruscan	Dead and classical	Etruscan + Latin	C	Known vs guessed, stated on every page
97	Ottoman Turkish	Dead and classical	Perso-Arabic + Latin	A	Assumes nothing; cross-links to Turkish, Arabic, Persian
98	Manchu	Dead and classical	Manchu + Latin (Mollendorff)	A	
99	Classical Mongolian	Dead and classical	Mongolian vertical + Latin	A	
100	Classical Tibetan	Dead and classical	Tibetan + Wylie	A	

Writing order: a pilot of six first (Latin, Georgian, Ge'ez, Turkish, Navajo, Classical Nahuatl), chosen to stress-test the template on different language types. After the pilot passes its gate (section 9), work through the dead and classical group, then isolates, then the rest by number.

3. Pedagogy: what "Tae Kim style" means

Tae Kim's guide teaches Japanese on its own terms: it starts from how the language actually builds meaning, orders topics so each one rests only on earlier ones, and explains the reason behind every form. Apply that method to every language. These are the writing rules.

Teach the language's own logic. Do not map it onto English grammar. If the language has no subject category, no tense, or no adjectives as a separate class, say so early and build from what it does have. Name the organising idea of each language in the overview (for example root-and-pattern morphology for Semitic, verb templates for Navajo, ergativity for Basque) and return to it throughout.

Strict dependency order. Each lesson uses only grammar already taught. Maintain a dependency graph per guide (section 4) and check every example against it. The order follows what a learner needs to build real sentences soonest, not the order of a reference grammar.

Why before how. Every rule gets a reason: a historical cause, a sound change, a functional pressure, or the system it belongs to. Where the true reason is unknown or disputed, say that instead of inventing one. Irregular forms are explained by their history wherever the history is known.

The chain of thought. For each construction, show how a speaker gets from meaning to form step by step. "I want to say X. The verb is Y. X is past, so… The object is definite, so…" The reader should be able to rebuild the sentence, not just recognise it.

Ground up, no dumbing down. Assume zero knowledge of the language and zero linguistics jargon. Define every technical term in one plain sentence the first time it appears, then use it freely. Do not simplify the facts; simplify the route to them.

Tables come last. A paradigm table appears only after the pattern behind it has been explained in prose and shown in examples. The table is a reference, never the lesson.

Real language. Teach the language as actually used. For living languages that means natural spoken forms, with formal or written variants labelled. For dead languages, prefer attested sentences from real texts, cited by work and passage.

Resolve contradictions on the spot. When a new rule seems to conflict with an earlier one, name the conflict and resolve it before moving on.

Connections. Each lesson may end with a short "Connections" note linking to another language in the app (cognates, loanwords, shared structures, typological parallels). Useful anchors for this reader: Spanish, Korean, Japanese, French, Arabic. Keep these notes short and accurate.

Check by doing. Each lesson ends with 5 to 10 exercises that ask the reader to build or take apart sentences, with worked answers that show the reasoning. No multiple choice, no flashcard drills.

Voice. Direct, plain, second person, British spelling. No filler, no encouragement padding, no "simply" or "just". Sentences short. One idea per paragraph.

Vocabulary is taught through the lessons, not as lists to memorise. Each lesson introduces 15 to 30 words chosen by frequency and by what the grammar point needs. Each guide ends with a core vocabulary of 800 to 1,500 words (smaller for depth B and C) with the lesson where each first appears.

4. Guide anatomy

Every guide follows one spine of nine parts, adapted to the language. Parts that do not apply are dropped (no case system, no case part) and language-specific parts are added where the language needs them (tone, clicks, noun classes, evidentials, speech levels, verb templates).

Part	Contents	Typical lessons
0. Orientation	Who spoke or speaks it, when, where; family; the organising idea; what makes it unusual; how to use the guide; a source note	2 to 3
1. Sounds and script	Sound inventory with IPA and plain descriptions; the writing system built up in stages; transliteration scheme used in the guide	4 to 10
2. The simplest sentences	Stating that something is something; basic word order; negation; yes/no questions	5 to 8
3. Nouns and their machinery	Number, gender or class, case or particles, definiteness, possession, pronouns	8 to 14
4. Verbs: the core system	Person marking, tense, aspect, mood, the main verb classes, how verbs are built	12 to 20
5. Modifying and linking	Adjectives, adverbs, numbers, comparison, conjunctions, postpositions or prepositions	8 to 12
6. Complex sentences	Relative clauses, subordinate clauses, conditionals, reported speech, questions with question words	8 to 12

Part	Contents	Typical lessons
7. Advanced and special topics	Voice, causatives, honorifics, evidentiality, registers, dialects, anything the language is known for	8 to 15
8. Reading real texts	5 to 10 graded authentic passages, each fully glossed and walked through line by line	5 to 10
9. Reference	Paradigm tables, core vocabulary, glossary of terms, annotated bibliography	appendices

For script-heavy languages, part 1 teaches the script in full. Examples carry transliteration throughout the guide, with a per-language app setting to hide it. For cuneiform languages and Classical Maya, the guide works mainly in transliteration and includes a proper introduction to how the script works.

Lesson template

Each lesson is 500 to 1,200 words and contains, in this order:

1. **Title and one-line point.** What the reader can do after this lesson.
2. **The idea.** The concept and why the language does it this way.
3. **How it is built.** The step-by-step chain from meaning to form.
4. **Examples.** 6 to 12 glossed examples, simplest first, each with a one-line note on what it shows.
5. **Watch out.** Apparent contradictions with earlier lessons, common misreadings, exceptions and their causes.
6. **Reference table.** Only if the lesson has a paradigm.
7. **New words.** The 15 to 30 words introduced.
8. **Connections.** Optional, short.
9. **Exercises and worked answers.**

File format

One folder per language under `content/<iso-code>/`. Lessons are Markdown files with YAML front matter. Examples use a fenced block so the app can render aligned interlinear glosses.

```
---
id: kat-04-07
lang: kat
part: 4
order: 7
title: The aorist: saying what happened
point: Form the simple past and understand why the subject changes case
requires: [kat-03-02, kat-04-03]
introduces: [aorist, ergative-case]
confidence: verified
sources: [aronson1990, hewitt1995]
---

## The idea
...

:::example
orig: (sentence in original script)
translit: (transliteration)
morphemes: (segmented words, hyphen-separated)
gloss: (Leipzig-style gloss, aligned word for word)
literal: (literal translation, if useful)
translation: (natural English)
note: (what this shows)
source: constructed | (citation for attested)
confidence: verified
:::
```

Glosses follow the Leipzig Glossing Rules. Every abbreviation used is listed in the guide's glossary and explained in plain words at first use.

Each language folder also holds:

- `meta.json`: name, endonym, ISO 639-3 code, family, region, era, status (living, endangered, extinct, classical), scripts, typology tags (word order, morphology type, alignment), depth class, one-paragraph hook, difficulty note for an English speaker.
- `syllabus.json`: ordered lesson list with ids, titles, `requires` and `introduces`. This is the dependency graph.
- `vocab.json`: headword, transliteration, part of speech, meaning, first lesson, confidence.
- `sources.json`: full bibliography with what each source was used for.
- `STATUS.json`: per-lesson state (section 6).

5. Accuracy protocol

Accuracy is the main risk of this project. A language model writes fluent, confident and wrong paradigms for low-resource languages. The owner will also use some of these guides as a base for reading real manuscripts, so errors compound. Treat your own recall as a draft to be checked, never as a source.

Source dossier first

Before writing any lesson for a language, build `sources.json` and a `dossier.md`:

1. Find the standard descriptive grammars, teaching grammars and dictionaries. Record author, year, title and what each is authoritative for.
2. Collect open, checkable sources: Wiktionary entries and inflection tables, WALS and Grambank feature values, Glottolog, Wikipedia grammar articles (as pointers to sources, not as authority), university course notes, open-access grammars (Language Science Press, institutional repositories), and digitised public-domain grammars.
3. For dead languages, locate open text corpora with translations (for example Perseus for Latin and Greek, ETCSL and ORACC for Sumerian and Akkadian, TITUS, the Thesaurus Linguae Aegyptiae, the Comprehensive Aramaic Lexicon, CEToM for Tocharian).
4. Write in `dossier.md`: phoneme inventory, script and transliteration scheme chosen (with reason), the major paradigms with their source, known points of scholarly disagreement, and dialect or period chosen as the base.
5. If web access is unavailable in a session, do not write lessons for a depth B or C language in that session. Work on depth A languages with strong recall, and mark everything `unverified` for a later pass.

Confidence labels

Every example and every table carries one label. The app shows it as a small coloured dot the reader can tap.

Label	Meaning	Rule
`verified`	Checked against at least one named source in this session	Record the source id
`attested`	A real sentence from a real text	Cite work and passage
`constructed`	Built by you from verified rules and verified vocabulary	Each morpheme must be individually checkable

Label	Meaning	Rule
`unverified`	Written from recall, not yet checked	Must be cleared by the verification pass before a guide is marked done
`uncertain`	Sources disagree or the form is reconstructed	Say so in the lesson text

A guide cannot reach status `done` with any `unverified` items in parts 1 to 6. For depth B and C languages, prefer attested examples over constructed ones.

Verification pass

After each part of a guide is drafted, run a separate checking pass, ideally as a fresh subagent that has not seen the drafting:

1. **Form check.** For every inflected form in examples and tables, look it up or re-derive it from the source paradigm. Fix or downgrade the label.
2. **Gloss check.** Morpheme count matches gloss count for every word. Abbreviations are in the glossary. Translation matches gloss.
3. **Dependency check.** A script reads `requires` and `introduces` and fails any example whose gloss uses a category not yet introduced.
4. **Consistency check.** The same word has the same transliteration and meaning everywhere. Run it against `vocab.json`.
5. **Script check.** Original-script text round-trips with the transliteration under the scheme in the dossier. Check Unicode normalisation and right-to-left marks.
6. **Claim check.** Historical and comparative claims (etymologies, "this comes from…") are sourced or removed.

Write the results to `content/<code>/VERIFY.md` with counts of items checked, corrected and still open.

Honesty rules for hard cases

- **Fragmentary languages** (Etruscan, parts of Sumerian, Tocharian B, Old Nubian, Classical Maya): separate "established", "probable" and "unknown" explicitly. A short honest guide is correct; a padded one is a failure.
- **Languages with major dialect splits** (Arabic varieties, Tamazight, Quechua, Kurdish, Mongolian): name the variety taught, and flag forms from other varieties.
- **Moroccan Darija:** teach it as spoken, in Arabizi first. Flag any form that is MSA or another dialect. Note Casablanca/Rabat versus northern variation and say when you are unsure.

- **Community-sensitive material** (Cherokee, Navajo, Lakota, Warlpiri, Dyirbal, Maori, Hawaiian): use only published teaching and descriptive material. Do not reproduce ceremonial or restricted texts. Credit community language programmes in the bibliography.
- **Magic and ritual corpora** (Ge'ez scrolls, incantation bowls, Coptic magical texts, muga): fine to teach from published editions as linguistic material, cited to the edition.

6. Production pipeline

The full library is roughly 4.5 million words of finished text plus verification, so it is built over many unattended sessions, not one. Each session must leave the repo in a clean, resumable state. Plan for usage limits to end a session without warning.

State files

- `PROGRESS.md` at the repo root: one row per language with stage, lessons done out of planned, word count, open `unverified` count, last session date.
- `content/<code>/STATUS.json`: one entry per lesson with state `planned`, `drafted`, `verified` or `done`.
- `DECISIONS.md`: every judgement call, dated, one line each.
- `SESSION_LOG.md`: appended at the end of each session: what was done, what failed, what to do next.

Stages per language

1. **Dossier.** Sources, transliteration scheme, base variety (section 5).
2. **Syllabus.** Full ordered lesson list with dependencies in `syllabus.json`. Check it against the spine in section 4 and against the standard grammar's table of contents so nothing major is missing.
3. **Draft.** Write lessons in order, one file per lesson. Commit after every lesson.
4. **Verify.** Run the verification pass per part. Commit corrections.
5. **Texts.** Write part 8 (graded readings) last, since it depends on everything.
6. **Reference.** Generate tables, vocabulary and glossary from the lesson files by script, then review by hand.
7. **Lint and mark done.** `npm run lint:content` must pass (section 9).

Session loop

At the start of every session:

1. Read this brief, `PROGRESS.md`, and the last entry of `SESSION_LOG.md`.

2. Pick the first language in the writing order that is not `done`, and its first unfinished stage.
3. Work in units of one lesson. After each: update `STATUS.json`, run the content linter on that file, commit with message `<code>: lesson <id> <state>`.
4. Every 10 lessons, update `PROGRESS.md`.
5. Before the session ends, or when context is running low, write the `SESSION_LOG.md` entry and commit.

Using subagents

If subagents are available, the main agent owns the syllabus and the state files and delegates:

- One drafting subagent per part of a guide, given the dossier, the syllabus, the lesson template and the already-written earlier parts.
- One verification subagent per part, given only the dossier, sources and the drafted files, with web access.
- Never run two drafting agents on the same language part. Parallelise across languages instead, at most three at a time, so quality can be watched.

Quality bar before scaling

Do not start language 7 until the six pilot guides pass the gate in section 9. If the pilot shows the template needs changing, change it, record it in `DECISIONS.md`, and update this brief's copy in the repo (`BRIEF.md`) so later sessions follow the new version.

Things that should stop a session

- A paid service or sign-up is required.
- The Apple ID or Apple Developer account is needed.
- Sources for a language cannot be reached and it is depth B or C. Skip to the next language and log it.

7. The iOS app

Version 1 ships as an installable web app (a PWA) that the owner adds to his iPhone home screen from Safari. It must work fully offline and must not look or feel like a website: from the home screen it should be indistinguishable from a native iOS reading app. The codebase is React Native through Expo, in TypeScript, so the same code can later be built

as a true native iOS app with no rewrite. Development happens entirely on a Windows laptop (Surface Laptop 3). Check the current Expo SDK version and its docs before scaffolding; do not rely on remembered API details.

Technical choices

- **Framework:** Expo (managed workflow), latest stable SDK, Expo Router, TypeScript strict mode. Web is the shipping target for version 1; iOS native must keep compiling.
- **Cross-platform discipline:** use only libraries that support both web and iOS. Platform-specific code goes in `.web.tsx` / `.ios.tsx` files. Never use DOM APIs outside `.web` files.
- **Content loading:** a build script (`npm run build:content`) converts `content/` Markdown into per-language JSON bundles in `app-content/`, plus one small `index.json` with all `meta.json` data. The app loads `index.json` at launch and a language's bundle only when opened. Never parse all 100 bundles at start-up.
- **Offline:** a service worker (Workbox, run after `npx expo export -p web`) precaches the app shell, fonts, `index.json` and every language bundle on first launch. Show a quiet one-time progress indicator ("Downloading library for offline use") and a settings line confirming everything is stored. After that, the app must work in airplane mode with no difference. Content updates download in the background and apply on next launch.
- **Local storage:** IndexedDB for progress, bookmarks, notes and settings. Home-screen web apps on iOS keep their storage, but add an export and import of user data as a JSON file in Settings as a backup.
- **Hosting:** a free static host (Vercel, Netlify or GitHub Pages). No backend, no analytics, no network calls except fetching the app's own files.
- **Rendering:** a custom lesson renderer for headings, prose, tables and the `:::example` block. No raw HTML injected from content.
- **Fonts:** bundle Noto web fonts, subset per script, loaded only when a language using that script is opened (but precached). Test each of these in the installed app on a real iPhone: Mandaic, Syriac, Ethiopic, Coptic and Old Nubian letters, Gothic, Glagolitic, Tifinagh, Balinese, Cherokee, Manchu and traditional Mongolian (vertical), Egyptian hieroglyphs, cuneiform, Tibetan stacks, Burmese, Khmer. Record results in `docs/script-rendering.md` .
- **Right-to-left:** Arabic, Hebrew, Syriac, Mandaic, Persian, Pashto, Ottoman and Urdu text must render right to left inside left-to-right lessons, with correct alignment in gloss blocks.
- **Maya glyphs:** there is no usable Unicode. Use only images that are public domain or openly licensed, or original simplified drawings, and record each licence.

Making it feel native, not like a website

This is a hard requirement. Test every item in the installed home-screen app on an iPhone, not in desktop Chrome.

- **Standalone mode:** web manifest with `display: standalone`, Apple meta tags for home-screen apps, `viewport-fit=cover`, and correct safe-area padding (`env(safe-area-inset-*)`) around the notch and home indicator. No browser bar, ever.
- **Icon and launch:** proper app icon at all Apple sizes and launch splash screens for current iPhone sizes, matching the app background so there is no white flash.
- **iOS look:** system font stack (SF Pro via `-apple-system`), iOS-style large titles that shrink on scroll, a bottom tab bar (Atlas, Shuffle, Library, Settings), translucent blurred bars, iOS-style grouped lists and bottom sheets.
- **Navigation:** push and pop transitions that slide like iOS, and an edge-swipe-back gesture, because standalone mode has no browser back button. Scroll position is kept when going back.
- **Kill web tells:** no blue underlined links, no hover states, no grey tap highlight (`-webkit-tap-highlight-color: transparent`), no long-press link previews or callouts on interface elements, no text selection on buttons and bars (lesson text stays selectable), no double-tap zoom (`touch-action: manipulation`), no page-level rubber-band bounce revealing a blank background (`overscroll-behavior: none` on the root, scroll inside containers), inputs at 16px or larger so Safari never zooms.
- **Speed:** no loading spinners for local content. Screens render instantly from cache. Animations at 60fps; test on the owner's actual phone.
- **Dark mode first** with a light mode, following the system setting by default. Status bar colour matches the theme.

Write `docs/NATIVE_FEEL_CHECKLIST.md` with each item above as a checkbox, and tick them only after testing in the installed app.

Screens

Screen	What it does
Atlas (home)	Scrollable list of all 100 languages with endonym in native script, family, status badge and progress ring. Search, filter (group, family, living or dead, script, typology tag) and sort
Shuffle	A large "Surprise me" button that picks a random language with an animated reveal. Options: any, dead only, never opened, same family as last

Screen	What it does
Language home	Hook paragraph, key facts, "the organising idea", typology tags, sample sentence with gloss, contents by part, continue button
Lesson reader	The lesson, with tap-to-expand glosses, transliteration toggle, text size control, previous and next, mark as read
Reference	Paradigm tables, searchable core vocabulary, glossary of terms, bibliography
Compare	Pick two languages and see their meta, typology tags and Swadesh-style core words side by side
Library	Bookmarks, highlights, personal notes, recently opened, overall stats
Settings	Theme, text size, transliteration default per language, show confidence dots, reset progress

The example block

This component matters most. It shows the original script large, the transliteration under it, then word-aligned columns of segmented morphemes over their glosses, then the translation. It must wrap cleanly on a narrow phone screen, keep each word's morphemes and gloss together, and handle right-to-left originals. Tapping a gloss abbreviation shows its plain-English meaning. Tapping the confidence dot shows the label and source.

Design

Dark mode first, with a light mode. Calm and bookish: generous line height, one accent colour per language group, no gamification, no streaks, no notifications. It should feel like a well-made reference book that happens to be on a phone. Load the frontend design guidance available to you before building screens.

Out of scope for version 1

Audio, spaced repetition, AI chat, accounts, sync, Android polish, App Store release.

8. Getting it onto the iPhone

Version 1 is a home-screen web app: free, permanent, offline, no Apple account and no Mac. A true native build stays possible later from the same code.

Stage	What the owner gets	Cost
1. Home-screen web app (version 1)	App icon on the home screen, opens full-screen, works offline forever, updates itself	Free
2. Native iOS app (optional, later)	The same app compiled as native iOS through EAS Build	Apple Developer Program, about £79 a year

Stage 1: deploy and install (do this)

1. Deploy the web export to a free static host and give the site a stable URL.
2. Write `docs/INSTALL_ON_IPHONE.md` in plain steps: open the URL in Safari, tap Share, then Add to Home Screen, open it once on Wi-Fi and wait for the library download to finish, then test in airplane mode.
3. Note in that file that it must be added from Safari, and that the app keeps its data as long as it stays on the home screen. Point to the backup export in Settings.
4. During development, also support previewing on the iPhone in Safari over the local network, documented in the same file.

Stage 2: native build (prepare only)

Do not build or pay for anything. Keep the iOS target compiling, add an `eas.json` with a `production` profile, and write `docs/NATIVE_LATER.md` explaining that a native build needs the paid Apple Developer Program when built from Windows through EAS, with the steps verified against current Expo docs.

Do not suggest unofficial sideloading tools, hackintosh setups or macOS virtual machines.

9. Repo layout, milestones and acceptance criteria

Layout

```
polyglot-atlas/
  BRIEF.md              this document, kept current
  CLAUDE.md             10-line pointer: read BRIEF.md, PROGRESS.md,
SESSION_LOG.md first
  PROGRESS.md
  DECISIONS.md
  SESSION_LOG.md
  languages.json        the 100 languages: number, name, ISO code, group,
depth
```

```
  content/
    <iso-code>/
      meta.json  dossier.md  sources.json  syllabus.json
      vocab.json  STATUS.json  VERIFY.md
      lessons/<part>-<order>-<slug>.md
  tools/
    lint-content.ts     schema, gloss alignment, dependency order, word
counts
    build-content.ts    Markdown to per-language JSON bundles
    stats.ts            regenerates PROGRESS.md numbers
  app/                  Expo Router screens
  components/           ExampleBlock, LessonRenderer, ParadigmTable...
  app-content/          generated, not hand-edited
  assets/fonts/
  docs/                 INSTALL_ON_IPHONE.md, NATIVE_FEEL_CHECKLIST.md,
NATIVE_LATER.md, script-rendering.md
```

Milestones

#	Milestone	Done when
M0	Foundations	Repo scaffolded, `languages.json` complete, linter and build script working on a dummy lesson, state files in place
M1	One exemplar guide	Latin complete at depth A, verified, linter passing. This is the reference for all others
M2	Pilot of six	Georgian, Ge'ez, Turkish, Navajo and Classical Nahuatl added. Template revised from what the pilot taught
M3	App version 1	Installed on the iPhone home screen with the six pilot guides, working in airplane mode, native-feel checklist ticked, INSTALL `_ON_IPHONE.md` written
M4	Dead and classical	Languages 72 to 100 done
M5	Isolates and Americas	Languages 59 to 71 done
M6	The rest	Languages 1 to 58 done
M7	Standalone build ready	`iOS target compiling, eas.json and NATIVE_LATER.md ready in case the owner goes native`

M3 comes early on purpose: the owner can start reading on his phone while the remaining guides are written. From M3 onward, every finished guide is rebuilt into `app-content/` in the same session.

Pilot gate (must pass before M4)

For each of the six pilot guides:

- Word count is inside its depth class range, and no lesson is under 400 words.
- Zero `unverified` items in parts 1 to 6. `VERIFY.md` shows every part checked.
- The linter passes: front matter valid, every example has all required fields, morpheme and gloss counts align, no example uses a category before its lesson.
- A cold read of lessons 1 to 10 by a fresh subagent, acting as a learner with no knowledge of the language, finds no undefined term and no unexplained form.
- Ten randomly sampled inflected forms per guide are re-checked against sources by a fresh subagent with zero errors. Any error triggers a full re-verification of that part.
- Original-script text renders correctly in the installed home-screen app, including right-to-left.

Definition of done for the project

All 100 guides at status `done`, `PROGRESS.md` showing zero open `unverified` in parts 1 to 6, the installed home-screen app running all 100 with search, shuffle, reader, reference and compare working offline, and both owner-facing docs written.

First session checklist

1. Create the repo and save this brief as `BRIEF.md`.
2. Write `languages.json` from section 2 with ISO 639-3 codes (look them up; several here need care, for example Hindi-Urdu, Serbo-Croatian, Tamazight variety, Classical Maya).
3. Build the linter and content build script.
4. Start M1: Latin dossier, syllabus, then lessons in order.
5. End with a `SESSION_LOG.md` entry.

Sources

Build and signing facts in section 8 were checked on 11 November 2026 against:

- Expo: Apple Developer Program explained, free vs paid
- Expo docs: Create a development build

- Expo: How to test an iOS app without a Mac

Method references for the content, to be consulted by the builder, not copied: Tae Kim's Guide to Learning Japanese (for approach only), the Leipzig Glossing Rules, WALS, Grambank, Glottolog and Wiktionary.